

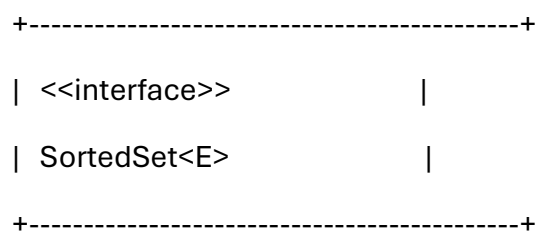
Aim

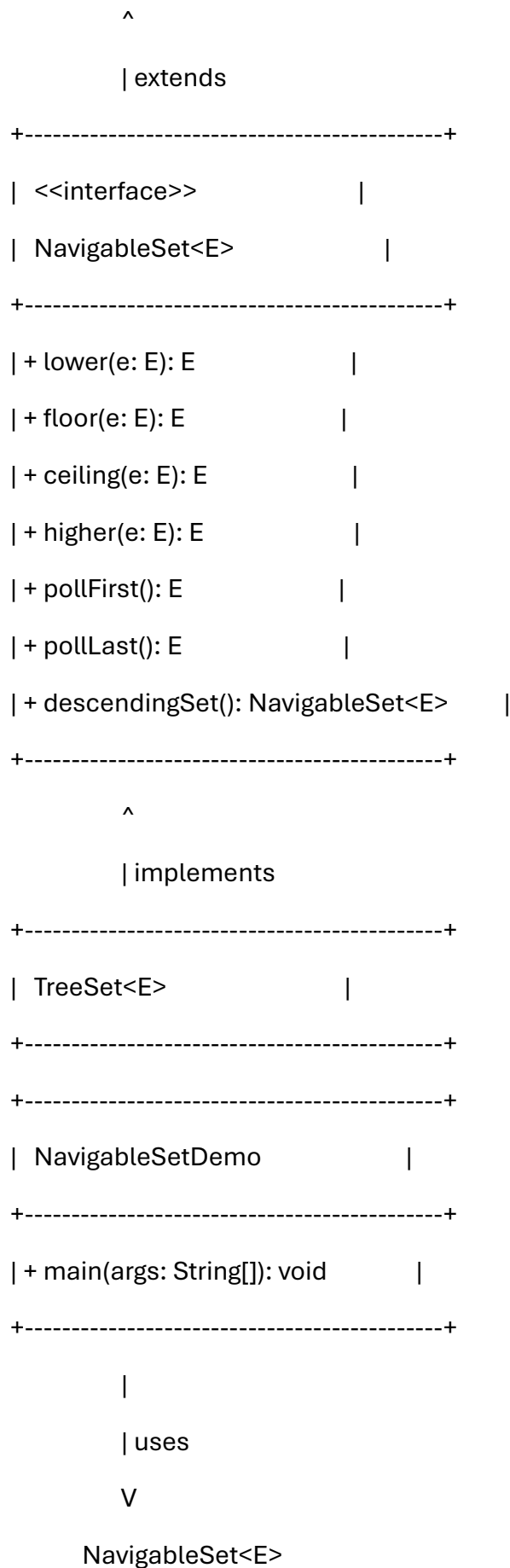
To write a Java program that demonstrates the methods of the NavigableSet interface, namely lower(), floor(), ceiling(), higher(), pollFirst(), pollLast() and descendingSet(), using a TreeSet to find the closest elements to a given value, remove the first and last elements, and view the set in reverse order.

Algorithm:

1. Start.
2. Create a NavigableSet of Integer using TreeSet.
3. Add the elements 30, 10, 50, 20 and 40 and display the set. It prints in sorted order.
4. Call lower(30) to get the greatest element strictly less than 30.
5. Call lower(10). It returns null because no smaller element exists.
6. Call floor(30) and floor(25) to get the greatest element less than or equal to the given value.
7. Call ceiling(30) and ceiling(25) to get the least element greater than or equal to the given value.
8. Call higher(30) to get the least element strictly greater than 30.
9. Call higher(50). It returns null because no larger element exists.
10. Call descendingSet() to display the elements in reverse order.
11. Call pollFirst() and pollLast() to retrieve and remove the lowest and highest elements.
12. Display the set again to show that both elements were removed.
13. Create an empty TreeSet and call pollFirst() on it. It returns null.
14. Stop.

UML Diagram:





Code:

```
package Observations;
```

```
import java.util.NavigableSet;
```

```
import java.util.TreeSet;
```

```
public class NavigableSetInterface {
```

```
    public static void main(String[] args) {
```

```
        NavigableSet<Integer> set = new TreeSet<>();
```

```
        set.add(30);
```

```
        set.add(10);
```

```
        set.add(50);
```

```
        set.add(20);
```

```
        set.add(40);
```

```
        System.out.println("Set: " + set);
```

```
        System.out.println("lower(30): " + set.lower(30));
```

```
        System.out.println("lower(10): " + set.lower(10));
```

```
        System.out.println("floor(30): " + set.floor(30));
```

```
        System.out.println("floor(25): " + set.floor(25));
```

```
        System.out.println("ceiling(30): " + set.ceiling(30));
```

```
        System.out.println("ceiling(25): " + set.ceiling(25));
```

```
        System.out.println("higher(30): " + set.higher(30));
```

```
        System.out.println("higher(50): " + set.higher(50));
```

```
        System.out.println("descendingSet(): " + set.descendingSet());
```

```

System.out.println("pollFirst(): " + set.pollFirst());

System.out.println("pollLast(): " + set.pollLast());

System.out.println("Set after polling: " + set);

NavigableSet<Integer> empty = new TreeSet<>();

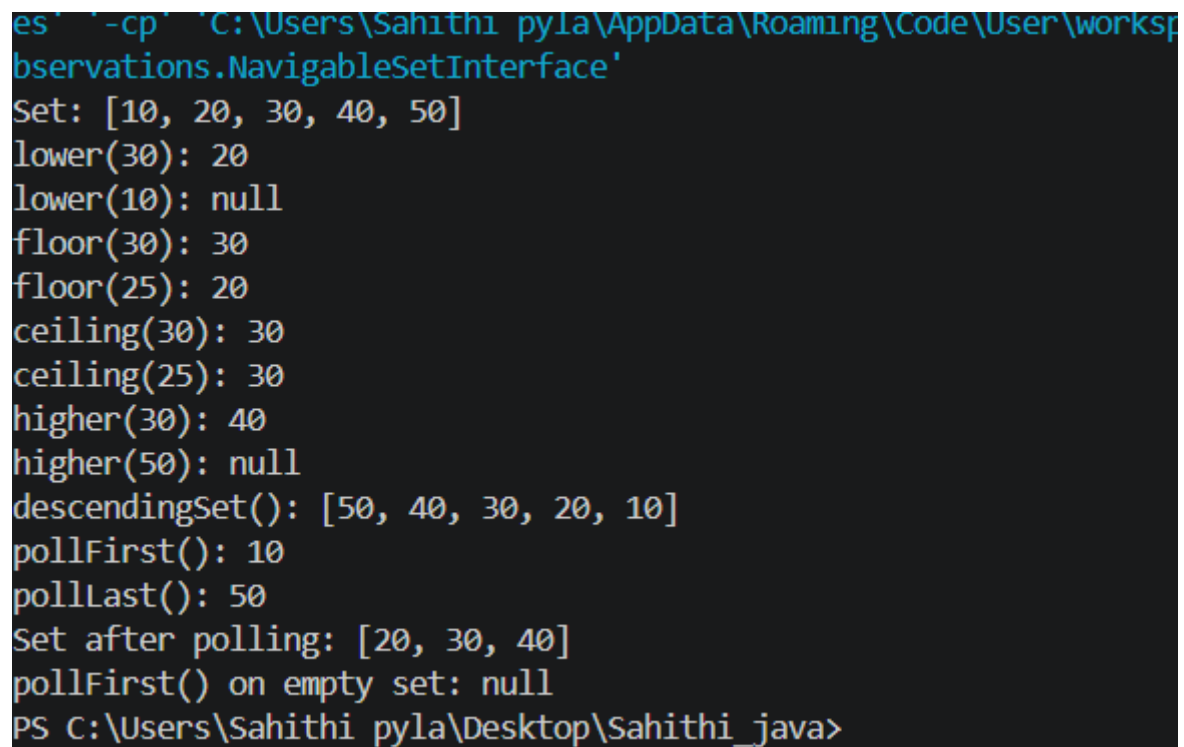
System.out.println("pollFirst() on empty set: " + empty.pollFirst());

}

}

```

Output:



```

es' -cp 'C:\Users\Sahithi_pyla\AppData\Roaming\Code\User\worksp
bservations.NavigableSetInterface'
Set: [10, 20, 30, 40, 50]
lower(30): 20
lower(10): null
floor(30): 30
floor(25): 20
ceiling(30): 30
ceiling(25): 30
higher(30): 40
higher(50): null
descendingSet(): [50, 40, 30, 20, 10]
pollFirst(): 10
pollLast(): 50
Set after polling: [20, 30, 40]
pollFirst() on empty set: null
PS C:\Users\Sahithi_pyla\Desktop\Sahithi_java>

```

Result:

The program was executed successfully. lower() and higher() excluded the given value, while floor() and ceiling() included it. pollFirst() and pollLast() removed the lowest and highest elements, and descendingSet() showed the elements in reverse order.